THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

DSAA2011 Machine Learning

L3 Supervised Learning: Regression and Classification II

Dr. Zixin Zhong

Data Science and Analytics Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

February 2, 2026

Course feedback

♡ Please scan the QR code and share your feedback.
No worries as it is anonymous. We sincerely appreciate your help.



Syllabus

Lec #	Topic	Lecturer
1	Introduction + course Info	Zixin
2-4	**Supervised learning: regression and classification**	Zixin
5	SVM (supervised learning) + Feature engineering	Zixin
6	Model evaluation and choice	Zixin
7	Ensemble learning (supervised learning) + Feature selection	Weikai
	Midterm (28 March): mark it on calendar!	
8-9	Unsupervised learning: clustering	Weikai
10	Dimension reduction	Weikai
11	Markov and graphical models	Weikai
12	Active learning + Reinforcement learning	Weikai
13	Project presentation	Both
	Project report submission (TBD)	
	Final exam (TBD)	



- Quiz:
  - Mark midterm time in your calendar!



Outline (L2 continued: from Slide 64)

1. Ridge regression
2. Gradient descent
3. Logistic regression



Recap of Lecture 2

- Least squares and linear regression
  - Linear regression with multiple outputs
  - Linear regression and MLE
- Linear classification
  - Linear models for binary classification
  - Linear models for multi-class classification
- Polynomial regression/classification



Linear regression

- **Learning/Training**: Given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, the least squares solution (with offset) is

$$\overline{\mathbf{w}}^* = \begin{bmatrix} b^* \\ \mathbf{w}^* \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}$$

where the design matrix and target vector are

$$\mathbf{X} = \begin{bmatrix} - \overline{\mathbf{x}}_1^\top - \\ - \overline{\mathbf{x}}_2^\top - \\ \vdots \\ - \overline{\mathbf{x}}_m^\top - \end{bmatrix} = \begin{bmatrix} 1 & - \mathbf{x}_1^\top - \\ 1 & - \mathbf{x}_2^\top - \\ \vdots & \vdots \\ 1 & - \mathbf{x}_m^\top - \end{bmatrix} \in \mathbb{R}^{m \times (d+1)} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \in \mathbb{R}^m.$$

- **Prediction/Testing**: Given a new feature vector (sample, example) $\mathbf{x}_{\text{new}}$, the prediction based on the least squares solution is

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{w}}^* = b^* + \mathbf{x}_{\text{new}}^\top \mathbf{w}^*.$$



Linear regression with multiple outputs

- Suppose there are h outputs we want to predict (above $h = 1$).
- Given a dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ (column vector) and $\mathbf{y}_i \in \mathbb{R}^{1\times h}$ (row vector), the model to be used is

$$\underbrace{\begin{bmatrix} y_{1,1} & y_{1,2} & \cdots & y_{1,h} \\ y_{2,1} & y_{2,2} & \cdots & y_{2,h} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m,1} & y_{m,2} & \cdots & y_{m,h} \end{bmatrix}}_{\mathbf{Y}\in\mathbb{R}^{m\times h}} = \underbrace{\begin{bmatrix} 1 & x_{1,1} & \cdots & x_{1,d} \\ 1 & x_{2,1} & \cdots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \cdots & x_{m,d} \end{bmatrix}}_{\mathbf{X}\in\mathbb{R}^{m\times(d+1)}} \underbrace{\begin{bmatrix} b_1 & b_2 & \cdots & b_h \\ w_{1,1} & w_{1,2} & \cdots & w_{1,h} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \cdots & w_{d,h} \end{bmatrix}}_{\overline{\mathbf{W}}\in\mathbb{R}^{(d+1)\times h}}$$

- When $h = 1$, this particularizes to standard linear regression.
- This is exactly h separate linear regression problems.



Linear regression with multiple outputs

- Suppose there are h outputs we want to predict (above $h = 1$).
- Given a dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^m$ where $\mathbf{x}_i \in \mathbb{R}^d$ (column vector) and $\mathbf{y}_i \in \mathbb{R}^{1\times h}$ (row vector), the model to be used is

$$\underbrace{\begin{bmatrix} y_{1,1} & y_{1,2} & \cdots & y_{1,h} \\ y_{2,1} & y_{2,2} & \cdots & y_{2,h} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m,1} & y_{m,2} & \cdots & y_{m,h} \end{bmatrix}}_{\mathbf{Y}\in\mathbb{R}^{m\times h}} = \underbrace{\begin{bmatrix} 1 & x_{1,1} & \cdots & x_{1,d} \\ 1 & x_{2,1} & \cdots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \cdots & x_{m,d} \end{bmatrix}}_{\mathbf{X}\in\mathbb{R}^{m\times(d+1)}} \underbrace{\begin{bmatrix} b_1 & b_2 & \cdots & b_h \\ w_{1,1} & w_{1,2} & \cdots & w_{1,h} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \cdots & w_{d,h} \end{bmatrix}}_{\overline{\mathbf{W}}\in\mathbb{R}^{(d+1)\times h}}$$

- When $h = 1$, this particularizes to standard linear regression.
- This is exactly h separate linear regression problems.



Linear regression with multiple outputs

- **Learning/Training**: Least Squares Solution

$$\overline{\mathbf{W}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y} \in \mathbb{R}^{(d+1)\times h}.$$

- **Prediction/Testing**: Given a new feature vector $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$, we can predict its h outputs as

$$\hat{\mathbf{y}}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{W}}^* \in \mathbb{R}^{1\times h}$$

- The k-th ($1 \le k \le h$) component of $\hat{\mathbf{y}}_{\text{new}}$ is the prediction of the k-th output based the dataset $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^m$.

◎ Is the matrix $\mathbf{X}^\top \mathbf{X}$ invertible?



MLE and linear regression

- Assume $y_i = \mathbf{w}^\top \mathbf{x}_i + b + e_i$ for each data point i and error $e_i \sim \mathcal{N}\left(0, \sigma^2\right)$.
- Likelihood function for the entire dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ is

$$L\left(\mathbf{W}, \sigma^2 \mid \{y_i, \mathbf{x}_i\}\right) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\left(y_i - \mathbf{W}^\top \mathbf{x}_i\right)^2}{2\sigma^2}\right)$$

- If $\mathbf{X}$ has full column rank, $\mathbf{X}^\top\mathbf{X}$ is invertible and the maximizer $(\hat{\mathbf{w}}, \hat{\sigma}^2)$ is:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \quad \text{(least squares solution)},$$

$$\hat{\sigma}^2 = \frac{1}{m}\sum_{i=1}^{m}(y_i - \left[1 \; x_i^\top\right] \cdot \hat{\mathbf{w}})^2 = \frac{1}{m}(\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}).$$

⊙ **MLE of distribution of error:** $e_i \sim \mathcal{N}(0, \hat{\sigma}^2) = \mathcal{N}(0, (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})/m)$.



MLE and linear regression

- If $\mathbf{X} = \begin{bmatrix} \overline{\mathbf{x}}_1^\top & \overline{\mathbf{x}}_2^\top & \dots & \overline{\mathbf{x}}_n^\top \end{bmatrix}^\top$ has full column rank, $\mathbf{X}^\top \mathbf{X}$ is invertible and

$$\frac{\partial}{\partial \overline{\mathbf{w}}} \log L\left(\overline{\mathbf{w}}, \sigma^2 \mid \{y_i, \mathbf{x}_i\}\right) = \frac{1}{\sigma^2} \sum_{i=1}^{m} \left(y_i - \overline{\mathbf{w}}^\top \overline{\mathbf{x}}_i\right) \overline{\mathbf{x}}_i = \mathbf{0}_{(d+1)\times 1} \tag{0.1}$$

$$\Rightarrow \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Proof. Firstly, (0.1) can be rewritten as $\sum_{i=1}^{m} \left(\overline{\mathbf{w}}^\top \overline{\mathbf{x}}_i\right) \overline{\mathbf{x}}_i = \sum_{i=1}^{m} y_i \overline{\mathbf{x}}_i$.



MLE and linear regression

Proof. Firstly, (0.1) can be rewritten as $\sum_{i=1}^{m}\left(\overline{\mathbf{w}}^{\top}\overline{\mathbf{x}}_i\right)\overline{\mathbf{x}}_i=\sum_{i=1}^{m}y_i\overline{\mathbf{x}}_i$. Then observe that

$$\sum_{i=1}^{m}y_i\overline{\mathbf{x}}_i=\begin{bmatrix}\overline{\mathbf{x}}_1 & \overline{\mathbf{x}}_2 & \dots & \overline{\mathbf{x}}_n\end{bmatrix}\cdot\begin{bmatrix}y_1\\ y_2\\ \vdots\\ y_n\end{bmatrix}=\mathbf{X}^{\top}\mathbf{y},$$

$$\sum_{i=1}^{m}\left(\overline{\mathbf{w}}^{\top}\overline{\mathbf{x}}_i\right)\overline{\mathbf{x}}_i=\sum_{i=1}^{m}\left(\overline{\mathbf{x}}_i^{\top}\overline{\mathbf{w}}\right)\overline{\mathbf{x}}_i=\sum_{i=1}^{m}\overline{\mathbf{x}}_i\left(\overline{\mathbf{x}}_i^{\top}\overline{\mathbf{w}}\right)=\sum_{i=1}^{m}\left(\overline{\mathbf{x}}_i\overline{\mathbf{x}}_i^{\top}\right)\overline{\mathbf{w}}=\left(\sum_{i=1}^{m}\overline{\mathbf{x}}_i\overline{\mathbf{x}}_i^{\top}\right)\overline{\mathbf{w}}$$

$$=\begin{bmatrix}\overline{\mathbf{x}}_1 & \overline{\mathbf{x}}_2 & \dots & \overline{\mathbf{x}}_m\end{bmatrix}\cdot\begin{bmatrix}\overline{\mathbf{x}}_1^{\top}\\ \overline{\mathbf{x}}_2^{\top}\\ \dots\\ \overline{\mathbf{x}}_m^{\top}\end{bmatrix}\cdot\overline{\mathbf{w}}=\mathbf{X}^{\top}\mathbf{X}\overline{\mathbf{w}}.$$



Then (0.1) can be further rewritten as $\mathbf{X}^{\top}\mathbf{X}\overline{\mathbf{w}}=\mathbf{X}^{\top}\mathbf{y}$.

MLE and linear regression: why Gaussian noise?

Theorem 0.1 (Central limit theorem (CLT))

Suppose $X_1, X_2, X_3 \ldots$ is a sequence of i.i.d. random variables with $\mathrm{E}[X_i] = \mu$ and $\mathrm{Var}[X_i] = \sigma^2 < \infty$. Then, as $n \to \infty$, the distribution of $\sqrt{n}\left(\bar{X}_n - \mu\right)$ converges to $\mathcal{N}\left(0, \sigma^2\right)$:

$$\sqrt{n}\left(\bar{X}_n - \mu\right) \xrightarrow{d} \mathcal{N}\left(0, \sigma^2\right)$$

- We say $X_n \xrightarrow{d} X$ as $n \to \infty$ if

$$\lim_{n\to\infty} F_{X_n}(x) = F_X(x) \ \forall x \text{ at which } F_X(x) \text{ is continuous.}$$

- In the case $\sigma > 0$, CLT implies that the cumulative distribution functions (cdf) of $\sqrt{n}\left(\bar{X}_n - \mu\right)$ converge pointwise to the cdf of the $\mathcal{N}\left(0, \sigma^2\right)$ distribution:

$$\lim_{n\to\infty} \mathbb{P}\left[\sqrt{n}\left(\bar{X}_n - \mu\right) \le z\right] = \lim_{n\to\infty} \mathbb{P}\left[\frac{\sqrt{n}\left(\bar{X}_n - \mu\right)}{\sigma} \le \frac{z}{\sigma}\right] = \Phi\left(\frac{z}{\sigma}\right)$$



where $\Phi(z)$ is the standard normal cdf evaluated at z.

Linear models for binary classification

- **Main idea:** to treat binary classification as regression where each label y_i can only take on -1 or $+1$.
- **Learning/Training:** given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ (where each $y_i \in \{+1, -1\}$), learn the weights using least squares

$$\overline{\mathbf{w}}^* = \begin{bmatrix} b^* \\ \mathbf{w}^* \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}.$$

- **Prediction/Testing:** given a new data sample $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$, its predicted label is

$$\hat{y}_{\text{new}} = \text{sign}\left(\overline{\mathbf{x}}_{\text{new}}^\top \overline{\mathbf{w}}^*\right) = \text{sign}\left(\begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{w}}^*\right) \in \{+1, -1\}.$$



Linear models for multi-class classification

- (Training/Learning) The design matrix $\mathbf{X}$ is the same. If it has full column rank, find the least squares solution

$$\overline{\mathbf{W}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y} \in \mathbb{R}^{(d+1)\times C}.$$

- (Testing/Prediction) Given a new feature vector $\mathbf{x}_{\text{new}} \in \mathbb{R}^d$, we have

$$\hat{y}^{\text{new,reg}}[:, k] = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{W}}^*[:, k] \quad \forall k \in \{1, 2, \ldots, C\}$$

and predict its class as

$$\hat{y}_{\text{new}} = \underset{k \in \{1,2,\ldots,C\}}{\arg\max} \left(\begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{W}}^*[:, k] \right) \in \{1, 2, \ldots, C\}$$



where $\overline{\mathbf{W}}^*[:, k] \in \mathbb{R}^{d+1}$ is the k-column of $\overline{\mathbf{W}}^*$.

Polynomial regression/classification

- **Learning/Training**:

$$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top)^{-1}\mathbf{y}$$

where

$$\mathbf{P} = \mathbf{P}(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m) = \begin{bmatrix} - \mathbf{p}_1^\top - \\ - \mathbf{p}_2^\top - \\ \vdots \\ - \mathbf{p}_m^\top - \end{bmatrix} \in \mathbb{R}^{m \times \binom{d+p}{p}}$$

- **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*.$$



Polynomial regression/classification

- For regression applications:
  - Learn continuous-valued y by using either primal or dual forms
  - Prediction:

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^{\top} \mathbf{w}^{*}.$$

- For classification applications:
  - Learn discrete-valued $y \in \{-1, +1\}$ (for binary classification) or one-hot encoded $\mathbf{Y}$ (for $y \in \{1, 2, \ldots, C\}$ for multi-class classification) using either primal or dual forms
  - Binary prediction

$$\hat{y}_{\text{new}} = \text{sign}\left(\mathbf{p}_{\text{new}}^{\top} \mathbf{w}^{*}\right)$$

  - Multi-class prediction

$$\hat{y}_{\text{new}} = \underset{k \in \{1,2,\ldots,C\}}{\arg\max} \left(\mathbf{p}_{\text{new}}^{\top} \mathbf{W}^{*}[:, k]\right)$$



Outline

1. Ridge regression
2. Gradient descent
3. Logistic regression



Motivation for ridge regression

How can we predict our academic performance in the coming semester?



Hours studied



Sleep hours



Extracurricular activities



Previous scores

Motivation for ridge regression

How can we predict our academic performance in the coming semester?



Hours studied



Sleep hours

- Subject
- Commute time
- Age
- Male/Female
- Family income
- .....



Extracurricular activities



Previous scores

Motivation for ridge regression

- This is the case of modern datasets which have many variables/attributes (d is large) and few samples (m is small).
- What happens to the least squares estimate?

$$\overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}?$$

Recall that this was obtained from minimizing

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})$$

over $\overline{\mathbf{w}} = \left[b, \mathbf{w}^\top \right]^\top \in \mathbb{R}^{d+1}$.



Motivation for ridge regression

- This is the case of modern datasets which have many variables/attributes (d is large) and few samples (m is small).
- What happens to the least squares estimate?

$$\overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}?$$

Recall that this was obtained from minimizing

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})$$

over $\overline{\mathbf{w}} = \left[b, \mathbf{w}^\top \right]^\top \in \mathbb{R}^{d+1}$.

- The design matrix $\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$ is very "wide".



Motivation for ridge regression

- This is the case of modern datasets which have many variables/attributes (d is large) and few samples (m is small).
- What happens to the least squares estimate?

$$\overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}?$$

Recall that this was obtained from minimizing

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})$$

over $\overline{\mathbf{w}} = \left[b, \mathbf{w}^\top \right]^\top \in \mathbb{R}^{d+1}$.

- The design matrix $\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$ is very "wide".
- Question: what is the coincidence?



Motivation for ridge regression

- This is the case of modern datasets which have many variables/attributes (d is large) and few samples (m is small).
- What happens to the least squares estimate?

$$\overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \in \mathbb{R}^{d+1}?$$

Recall that this was obtained from minimizing

$$J(\overline{\mathbf{w}}) = \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})$$

over $\overline{\mathbf{w}} = \left[b, \mathbf{w}^\top \right]^\top \in \mathbb{R}^{d+1}$.

- The design matrix $\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$ is very "wide".
- $\mathbf{X}$ is highly unlikely to have full column rank $\Longrightarrow$ $(\mathbf{X}^\top \mathbf{X})^{-1}$ does not exist.



Motivation for ridge regression

- Model possess too many features
- Go beyond the linear model, even an infinite-dimensional model



Motivation for ridge regression

- Model possess too many features
- Go beyond the linear model, even an infinite-dimensional model
- ◎ Stabilize and robustify the solution



New objective function for ridge regression

- Recap of linear regression: We average the square of the errors over all training samples. This defines the objective or loss function

$$\text{Loss}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 .$$



New objective function for ridge regression

- Recap of linear regression: We average the square of the errors over all training samples. This defines the objective or loss function

$$\text{Loss}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 .$$

- **Ridge regression:** For a fixed $\lambda \geq 0$, consider

$$\begin{aligned} J(\overline{\mathbf{w}}) &= \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 + \lambda \sum_{i=0}^{d} w_j^2 \\ &= (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} \end{aligned}$$

  Note that $w_0 = b$, the offset or bias.



New objective function for ridge regression

- **Ridge regression:** For a fixed $\lambda \geq 0$, consider

$$\begin{aligned} J(\overline{\mathbf{w}}) &= \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}_i) - y_i \right)^2 + \lambda \sum_{i=0}^{d} w_j^2 \\ &= (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} \end{aligned}$$

  Note that $w_0 = b$, the offset or bias.
- The term $\lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}$ encourages the weight vector to have small components (also known as shrinkage.
- The new objective results in ridge regression or Tikhonov regularization.
- When $\lambda = 0$, we recover usual linear regression.



Solution for ridge regression

- Recall that we wish to solve

$$\overline{\mathbf{w}}^* = \underset{\overline{\mathbf{w}}=[b,\mathbf{w}]^\top}{\arg\min} \ (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}.$$



Solution for ridge regression

- Recall that we wish to solve

$$\overline{\mathbf{w}}^* = \underset{\overline{\mathbf{w}}=[b,\mathbf{w}]^\top}{\arg\min} \ (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}.$$

- Expanding the objective, we obtain

$$\begin{aligned}(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} &= \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{X}\overline{\mathbf{w}} - \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\overline{\mathbf{w}} + \mathbf{y}^\top \mathbf{y} + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} \\ &= \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{X}\overline{\mathbf{w}} + \overline{\mathbf{w}}^\top (\lambda \mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y} \\ &= \overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y}\end{aligned}$$



Solution for ridge regression

- Recall that we wish to solve

$$\overline{\mathbf{w}}^* = \underset{\overline{\mathbf{w}}=[b,\mathbf{w}]^\top}{\arg\min} \ (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}.$$

- Expanding the objective, we obtain

$$\begin{aligned}(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} &= \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{X}\overline{\mathbf{w}} - \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\overline{\mathbf{w}} + \mathbf{y}^\top \mathbf{y} + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} \\ &= \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{X}\overline{\mathbf{w}} + \overline{\mathbf{w}}^\top (\lambda \mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y} \\ &= \overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y}\end{aligned}$$

- Differentiating w.r.t. $\overline{\mathbf{w}}$ and setting the result to zero yields

$$2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\overline{\mathbf{w}}^* = 2(\mathbf{X}^\top \mathbf{y}) \quad \Longleftrightarrow \quad \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$



Solution for ridge regression

- Recall that we wish to solve

$$\overline{\mathbf{w}}^* = \underset{\overline{\mathbf{w}}=[b,\mathbf{w}]^\top}{\arg\min} \ (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}}.$$

- Expanding the objective, we obtain

$$\begin{aligned}(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} &= \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{X}\overline{\mathbf{w}} - \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\overline{\mathbf{w}} + \mathbf{y}^\top \mathbf{y} + \lambda \overline{\mathbf{w}}^\top \overline{\mathbf{w}} \\ &= \overline{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{X}\overline{\mathbf{w}} + \overline{\mathbf{w}}^\top (\lambda \mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y} \\ &= \overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{y}\end{aligned}$$

- Differentiating w.r.t. $\overline{\mathbf{w}}$ and setting the result to zero yields

$$2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\overline{\mathbf{w}}^* = 2(\mathbf{X}^\top \mathbf{y}) \quad \Longleftrightarrow \quad \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

- For any $\lambda > 0$, $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is always invertible (why?) so the calculation above is legitimate.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda > 0)$ is always invertible

Why?



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda > 0)$ is always invertible

Proposition 1.1

The vector space consisting of only the zero vector has dimension 0.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda > 0)$ is always invertible

Proposition 1.1

The vector space consisting of only the zero vector has dimension 0.

Proof. Apply the definition of dimension.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda > 0)$ is always invertible

Proposition 1.1

The vector space consisting of only the zero vector has dimension 0.

Proof. Apply the definition of dimension.

Definition 1.2 (Definite matrix)

Let $\mathbf{A}$ denote a square matrix in $\mathbb{R}^{n\times n}$. $\mathbf{A}$ is said to be **positive-definite** if

$$\mathbf{x}^\top\mathbf{A}\mathbf{x} > 0 \text{ for all } \mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}.$$

$\mathbf{A}$ is said to be **negative-definite** if

$$\mathbf{x}^\top\mathbf{A}\mathbf{x} < 0 \text{ for all } \mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}.$$



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda > 0)$ is always invertible

Proposition 1.3

If $A \in \mathbb{R}^{n\times n}$ is positive-definite or negative-definite, then A is invertible.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proposition 1.3

If $A \in \mathbb{R}^{n\times n}$ is positive-definite or negative-definite, then A is invertible.

Proof. (I) If A is positive-definite, $\mathbf{x}^\top\mathbf{A}\mathbf{x} > 0$ for all $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ implies that

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = 0\} = \{\mathbf{0}\}. \tag{1.1}$$



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proposition 1.3

If $A \in \mathbb{R}^{n\times n}$ is positive-definite or negative-definite, then A is invertible.

Proof. (I) If A is positive-definite, $\mathbf{x}^\top\mathbf{A}\mathbf{x} > 0$ for all $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ implies that

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = 0\} = \{\mathbf{0}\}. \tag{1.1}$$

Hence, $\dim(\mathcal{N}(\mathbf{A})) = 0$



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proposition 1.3

If $A \in \mathbb{R}^{n\times n}$ is positive-definite or negative-definite, then A is invertible.

Proof. (I) If A is positive-definite, $\mathbf{x}^\top\mathbf{A}\mathbf{x} > 0$ for all $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ implies that

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = 0\} = \{\mathbf{0}\}. \tag{1.1}$$

Hence, $\dim(\mathcal{N}(\mathbf{A})) = 0$ and $\operatorname{rank}(\mathbf{A}) = \dim(\mathcal{R}(\mathbf{A})) = n - \dim(\mathcal{N}(\mathbf{A})) = n$.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proposition 1.3

If $A \in \mathbb{R}^{n\times n}$ is positive-definite or negative-definite, then A is invertible.

Proof. (I) If A is positive-definite, $\mathbf{x}^\top\mathbf{A}\mathbf{x} > 0$ for all $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ implies that

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = 0\} = \{\mathbf{0}\}. \tag{1.1}$$

Hence, $\dim(\mathcal{N}(\mathbf{A})) = 0$ and $\operatorname{rank}(\mathbf{A}) = \dim(\mathcal{R}(\mathbf{A})) = n - \dim(\mathcal{N}(\mathbf{A})) = n$. Therefore, A is invertible.
(II) Case where A is negative-definite can be similarly proven.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proof.

- $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is a square matrix.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proof.

- $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is a square matrix.
- To show $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is invertible, it is sufficient to show that $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is positive-definite or negative definite.



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proof.

- $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is a square matrix.
- To show $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is invertible, it is sufficient to show that $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is positive-definite or negative definite.
- For all $\mathbf{z} \in \mathbb{R}^{(d+1)\backslash\{\mathbf{0}\}}$,

$$\mathbf{z}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\mathbf{z} = \mathbf{z}^\top(\mathbf{X}^\top\mathbf{X})\mathbf{z} + \mathbf{z}^\top(\lambda\mathbf{I})\mathbf{z} = (\mathbf{X}\mathbf{z})^\top(\mathbf{X}\mathbf{z}) + \lambda\mathbf{z}^\top\mathbf{z} > 0. \tag{1.2}$$



Legitimacy: $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda > 0)$ is always invertible

Proof.

- $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is a square matrix.
- To show $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is invertible, it is sufficient to show that $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} \in \mathbb{R}^{(d+1)\times(d+1)}$ is positive-definite or negative definite.
- For all $\mathbf{z} \in \mathbb{R}^{(d+1)\backslash\{\mathbf{0}\}}$,

$$\mathbf{z}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\mathbf{z} = \mathbf{z}^\top(\mathbf{X}^\top\mathbf{X})\mathbf{z} + \mathbf{z}^\top(\lambda\mathbf{I})\mathbf{z} = (\mathbf{X}\mathbf{z})^\top(\mathbf{X}\mathbf{z}) + \lambda\mathbf{z}^\top\mathbf{z} > 0. \tag{1.2}$$

- Hence, $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ is positive-definite and hence invertible.



[Exercise] $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall \lambda \leq 0)$ is always invertible?

- We wish to solve

$$\overline{\mathbf{w}}^* = \underset{\overline{\mathbf{w}}=[b,\mathbf{w}]^\top}{\arg\min} \ (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda\overline{\mathbf{w}}^\top\overline{\mathbf{w}}.$$

- Expanding the objective, we obtain

$$(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda\overline{\mathbf{w}}^\top\overline{\mathbf{w}} = \overline{\mathbf{w}}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top(\mathbf{X}^\top\mathbf{y}) + \mathbf{y}^\top\mathbf{y}$$

- Differentiating w.r.t. $\overline{\mathbf{w}}$ and setting the result to zero yields

$$2(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\overline{\mathbf{w}}^* = 2(\mathbf{X}^\top\mathbf{y}) \quad \Longleftrightarrow \quad \overline{\mathbf{w}}^* = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}.$$

- For any $\lambda \leq 0$, $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ is always invertible?
- For any $\lambda < 0$, $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ is always invertible?
- Counter example?



Ridge regression in primal form

- **Training/Learning**: Minimizing the ridge regression objective $J(\overline{\mathbf{w}}) = (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda\overline{\mathbf{w}}^\top\overline{\mathbf{w}}$ yields

$$\overline{\mathbf{w}}^* = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}.$$

- **Testing/Prediction**: Given a new test sample $\mathbf{x}_{\text{new}}$, its prediction is

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{w}}^*.$$



Ridge regression in primal form

- The solution is known as the

$$\text{[Primal Form]} \qquad \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Use $\mathbf{I}_{d+1}$ to emphasize that the identity matrix is of size $(d+1) \times (d+1)$.



Ridge regression in primal form

- The solution is known as the

$$\text{[Primal Form]} \qquad \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}.$$

  Use $\mathbf{I}_{d+1}$ to emphasize that the identity matrix is of size $(d+1) \times (d+1)$.
- Q: What is the problem with inverting the $(d+1) \times (d+1)$ matrix $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$?



Ridge regression in primal form

- The solution is known as the

$$\text{[Primal Form]} \qquad \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}.$$

  Use $\mathbf{I}_{d+1}$ to emphasize that the identity matrix is of size $(d+1) \times (d+1)$.
- Q: What is the problem with inverting the $(d+1) \times (d+1)$ matrix $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$?
- $d > m$ is very large. Inverting the $(d+1) \times (d+1)$ matrix is not advisable!
- This takes $\approx d^3$ operations (multiplications and additions).
  [You don't need to know why.]



Ridge regression in primal form

- The solution is known as the

$$\text{[Primal Form]} \qquad \overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}.$$

  Use $\mathbf{I}_{d+1}$ to emphasize that the identity matrix is of size $(d+1) \times (d+1)$.
- Q: What is the problem with inverting the $(d+1) \times (d+1)$ matrix $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$?
- $d > m$ is very large. Inverting the $(d+1) \times (d+1)$ matrix is not advisable!
- This takes $\approx d^3$ operations (multiplications and additions).
  [You don't need to know why.]
- If $m > d$, we can still use

$$\overline{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}.$$



Ridge regression in dual form

- Fact: For every $\lambda > 0$,

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}. \tag{P-D}$$

- **Training/Learning:** So when $d > m$ (modern datasets), we use the

$$\text{[Dual Form]} \qquad \overline{\mathbf{w}}^* = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}.$$



Ridge regression in dual form

- Fact: For every $\lambda > 0$,

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}. \qquad \text{(P-D)}$$

- **Training/Learning:** So when $d > m$ (modern datasets), we use the

$$\text{[Dual Form]} \qquad \overline{\mathbf{w}}^* = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}.$$

- **Testing/Prediction:** Given a new test sample $\mathbf{x}_{\text{new}}$, its prediction is

$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \overline{\mathbf{w}}^*.$$

- To show (P-D), we use the Woodbury formula

$$(\mathbf{I} + \mathbf{U}\mathbf{V})^{-1} = \mathbf{I} - \mathbf{U}(\mathbf{I} + \mathbf{V}\mathbf{U})^{-1}\mathbf{V}.$$



Ridge regression in dual form [exercise]

Apply the Woodbury formula

$$(\mathbf{I} + \mathbf{UV})^{-1} = \mathbf{I} - \mathbf{U}(\mathbf{I} + \mathbf{VU})^{-1}\mathbf{V}$$

to show

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1}\mathbf{X}^\top \mathbf{y} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1}\mathbf{y}. \tag{P-D}$$



Ridge regression in dual form [exercise]

Note that $\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$. Starting from $\mathbf{X}^\top \left(\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m\right)^{-1} \mathbf{y}$, we have

$$
\begin{aligned}
& \mathbf{X}^\top \left(\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m\right)^{-1} \mathbf{y} \\
&= \lambda^{-1} \mathbf{X}^\top \left(\mathbf{I}_m + \lambda^{-1} \mathbf{X}\mathbf{X}^\top\right)^{-1} \mathbf{y} \\
&= \lambda^{-1} \mathbf{X}^\top \left[\mathbf{I}_m - \lambda^{-1} \mathbf{X} \left(\mathbf{I}_{d+1} + \lambda^{-1} \mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top\right] \mathbf{y} \qquad (1.3) \\
&= \lambda^{-1} \left(\mathbf{X}^\top \mathbf{y} - \mathbf{X}^\top \mathbf{X} \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}\right)^{-1} \mathbf{X}^\top \mathbf{y}\right) \\
&= \lambda^{-1} \left(\mathbf{I}_{d+1} - \mathbf{X}^\top \mathbf{X} \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}\right)^{-1}\right) \mathbf{X}^\top \mathbf{y} \\
&= \lambda^{-1} \left[\mathbf{I}_{d+1} - \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}\right) \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}\right)^{-1} + \lambda \mathbf{I}_{d+1} \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}\right)^{-1}\right] \mathbf{X}^\top \mathbf{y} \\
&= \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}\right)^{-1} \mathbf{X}^\top \mathbf{y}
\end{aligned}
$$

where (1.3) follows from the Woodbury matrix identity with $\mathbf{U} \equiv \lambda^{-1}\mathbf{X}$ and $\mathbf{V} \equiv \mathbf{X}^\top$.



Python demo: ridge regression

	Previous Scores	Hours Studied	Extracurricular Activities_bool	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	99	7	1	9	1	91.0
1	82	4	0	4	2	65.0
2	51	8	1	7	2	45.0
3	52	5	1	5	2	36.0
4	75	7	0	8	5	66.0
5	78	3	0	9	6	61.0
6	73	7	1	5	6	63.0
7	45	8	1	4	6	42.0
8	77	5	0	8	2	61.0
9	89	4	0	4	0	69.0
10	91	8	0	4	5	84.0
11	79	8	0	6	2	73.0
12	47	3	0	9	2	27.0
13	47	6	0	4	2	33.0
14	79	5	0	7	8	68.0



Python demo: ridge regression

```
X = df_bool[['Previous Scores',
 'Hours Studied',
 'Extracurricular Activities_bool',
 'Sleep Hours',
 'Sample Question Papers Practiced']].to_numpy(copy=True)
y = df_bool[['Performance Index']].to_numpy(copy=True)
```

```
# split the data into training and test samples
X_train, X_test, y_train, y_test = sklearn.model_selection.train_test_split(
    X, y, test_size=0.3)
clf = Ridge(alpha=1.0).fit(X_train, y_train)
```



Python demo: ridge regression

```python
for i in range(10):
    X_true = X_test[i,:].reshape(1, -1)
    y_true = y_test[i]
    y_pred = clf.predict(X_true)

    print('%d-th new sample:' % (i+1))
    print('True y: %.3f' % y_true[0])
    print('Predicted y: %.3f' % y_pred[0])
    print('=========')
```

```
1-th new sample:
True y: 36.000
Predicted y: 33.970
=========
2-th new sample:
True y: 26.000
Predicted y: 25.063
=========
```



Ridge regression (polynomial form)

- Ridge regression in primal form (when $m > d' = \binom{p+d}{p}$)
  - **Learning/Training**:
    $$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}$$
  - **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$
    $$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$
    where $\mathbf{p}_{\text{new}}$ is the polynomial vector associated to $\mathbf{x}_{\text{new}}$.

- Ridge regression in dual form (when $m < d' = \binom{p+d}{p}$)
  - **Learning/Training**:
    $$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$$
  - **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$
    $$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*.$$



Ridge regression (polynomial form)

- Primal Form
  - Learning/Training
$$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P} \mathbf{y}$$
  - Prediction/Testing
$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$
- Dual Form
  - **Learning/Training**:
$$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$$
  - **Prediction/Testing**:
$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$
- Useful Python packages and functions

  `sklearn.preprocessing PolynomialFeatures, np.sign, sklearn.model_selection train_test_split, sklearn.preprocessing OneHotEncoder, pandas`



Outline

1 Ridge regression

2 Gradient descent

3 Logistic regression



Motivation for gradient descent

- Polynomial regression: optimal $\mathbf{w}^*$ can be written with the following "closed-form" formula (primal solution):

$$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}.$$



Motivation for gradient descent

- Polynomial regression: optimal $\mathbf{w}^*$ can be written with the following "closed-form" formula (primal solution):

$$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}.$$

- In other model, finding the optimal parameter $\mathbf{w}$ (by minimizing some objective function $C(\mathbf{w})$) might not be so easy.

⊙ **How to solve this issue?**



Motivation for gradient descent

- Polynomial regression: optimal $\mathbf{w}^*$ can be written with the following "closed-form" formula (primal solution):

$$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}.$$

- In other model, finding the optimal parameter $\mathbf{w}$ (by minimizing some objective function $C(\mathbf{w})$) might not be so easy.

⊙ **How to solve this issue?**

- Usually have to estimate **w** iteratively with some algorithm
- Optimization workhorse for modern machine learning is **gradient descent**.



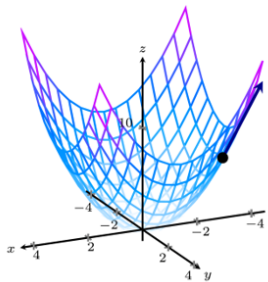
Gradient descent (GD) algorithm

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \ldots, w_d]^\top$



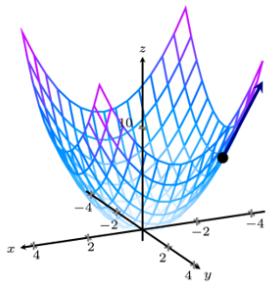
Gradient descent (GD) algorithm

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \ldots, w_d]^\top$
- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$



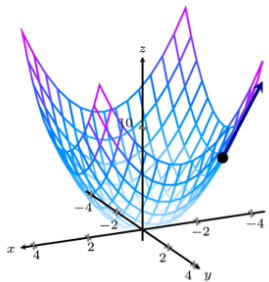
Gradient descent (GD) algorithm

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \ldots, w_d]^\top$
- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$
  - $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$



Gradient descent (GD) algorithm

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \ldots, w_d]^\top$
- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$



  - $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$
  - $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is increasing most rapidly
    $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly



Why?

- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$
- $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is increasing most rapidly
  $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly



Why?

- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$
- $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is increasing most rapidly
  $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly

⊙ Hint: Think about the case with $\mathbf{w} \in \mathbb{R}^1$.



Why?

- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$
- $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is increasing most rapidly
  $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly

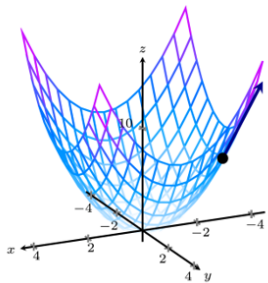
- Hint: Think about the case with $\mathbf{w} \in \mathbb{R}^1$.
- Book: 'Multivariable Calculus' by James Stewart



Gradient descent (GD) algorithm

Algorithm 1: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$



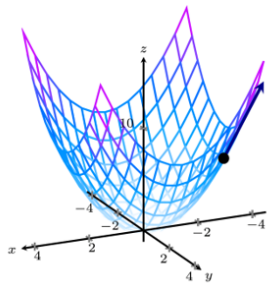


Gradient descent (GD) algorithm

Algorithm 1: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**

7: **end while**

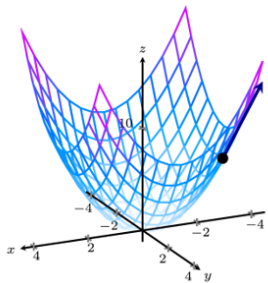


Gradient descent (GD) algorithm

Algorithm 1: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$

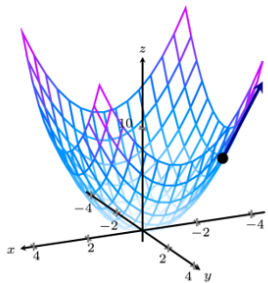
7: **end while**



Gradient descent (GD) algorithm

Algorithm 1: Gradient Descent (GD)

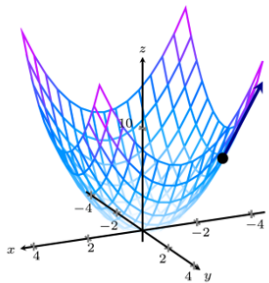
1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
4: **if** converge **then**
5: **return** $\mathbf{w}_{k+1}$
6: **end if**
7: **end while**



Gradient descent (GD) algorithm

Algorithm 1: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
4: **if** converge **then**
5: **return** $\mathbf{w}_{k+1}$
6: **end if**
7: **end while**



◦ According to multi-variable calculus, if η is not too big, then $C(\mathbf{w}_{k+1}) < C(\mathbf{W}_k)$.

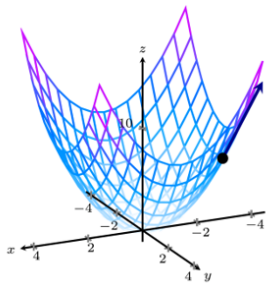
⇒ We get better $\mathbf{w}$ after each iteration.



Gradient descent (GD) algorithm

Algorithm 2: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
4: **if** converge **then**
5: **return** $\mathbf{w}_{k+1}$
6: **end if**
7: **end while**



◦ According to multi-variable calculus, if η is not too big, then $C(\mathbf{w}_{k+1}) < C(\mathbf{W}_k)$.

⇒ We get better $\mathbf{w}$ after each iteration.



Gradient descent (GD) algorithm

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold



Gradient descent (GD) algorithm

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold
- Gradient descent can only find local minimum



Gradient descent (GD) algorithm

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold

- Gradient descent can only find local minimum
  - **Why?**



Gradient descent (GD) algorithm

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold

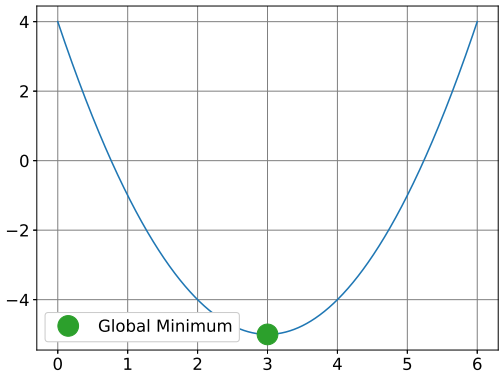
- Gradient descent can only find local minimum
  - Because gradient $= \mathbf{0}$ at local minimum
    so $\mathbf{w}$ won't change after that



Gradient descent (GD) algorithm

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold

- Gradient descent can only find local minimum
  - Because gradient $= \mathbf{0}$ at local minimum
    so $\mathbf{w}$ won't change after that

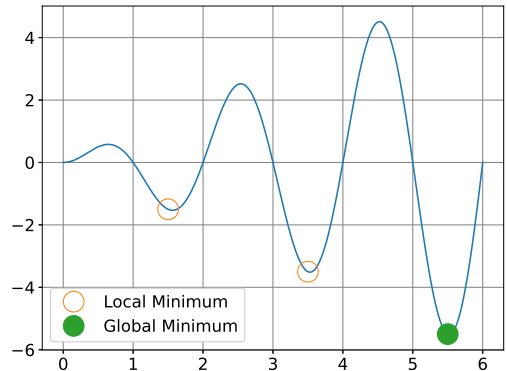




Gradient descent (GD) algorithm

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold

- Gradient descent can only find local minimum
  - Because gradient $= \mathbf{0}$ at local minimum
    so $\mathbf{w}$ won't change after that





Gradient descent (GD) algorithm

Any question so far?



GD: choice of convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in objective function C below a threshold

- Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold



GD: choice of convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in objective function C below a threshold
  - Pro: direct measure of optimization progress, prevents unnecessary iterations
  - Con: computational cost, sensitive to scale, NOT guarantee convergence of $\mathbf{w}$
- Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold



GD: choice of convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in objective function C below a threshold
  - Pro: direct measure of optimization progress, prevents unnecessary iterations
  - Con: computational cost, sensitive to scale, NOT guarantee convergence of $\mathbf{w}$
- Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold
  - Pro: stable solution, less sensitive to oscillations in the function value
  - Con: can take longer to detect convergence, NOT directly reflect progress in objective function



GD: choice of convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in objective function C below a threshold
  - Pro: direct measure of optimization progress, prevents unnecessary iterations
  - Con: computational cost, sensitive to scale, NOT guarantee convergence of $\mathbf{w}$
- Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold
  - Pro: stable solution, less sensitive to oscillations in the function value
  - Con: can take longer to detect convergence, NOT directly reflect progress in objective function

⊙ Change of function C or parameter $\mathbf{w}$?

- Function C: care primarily about minimizing C, C is well-behaved (e.g., smooth and convex), low computational cost
- Parameter $\mathbf{w}$: care primarily if the parameters are stable and the optimization process has converged

○ Do hybrid?



Example of GD: find x to minimize $g(x) = x^2$



Example of GD: find x to minimize $g(x) = x^2$

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent



Example of GD: find x to minimize $g(x) = x^2$

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$



Example of GD: find x to minimize $g(x) = x^2$

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$



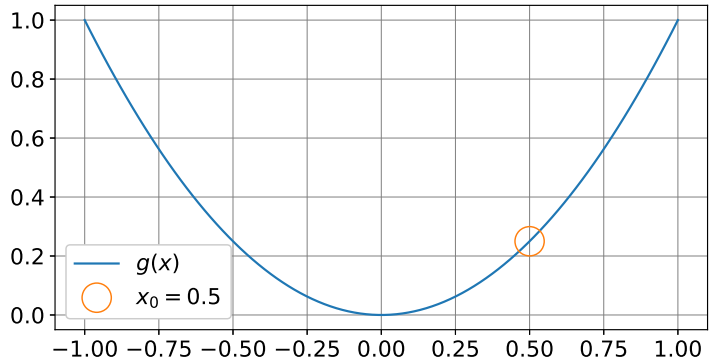
Example of GD: find x to minimize $g(x) = x^2$

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$



Example of GD: find x to minimize $g(x) = x^2$

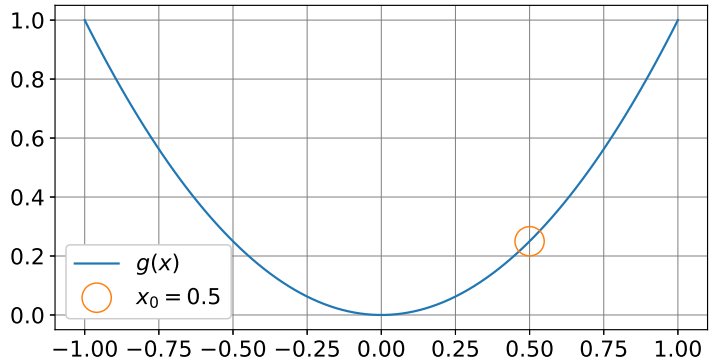
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_0 = 0.5$,





Example of GD: find x to minimize $g(x) = x^2$

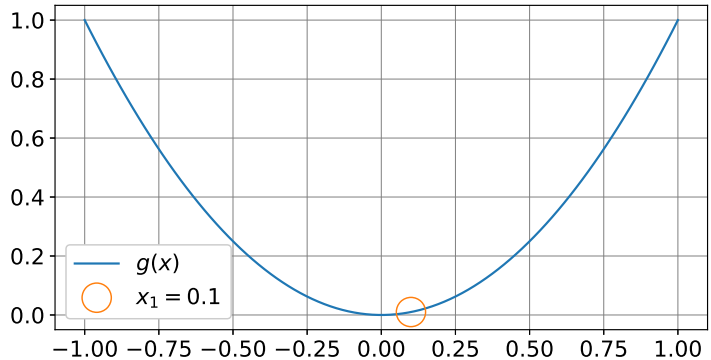
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_0 = 0.5, \quad \nabla_x g(x_0) = 1, \quad x_1 = x_0 - 0.4 \nabla_x g(x_0) = 0.1$





Example of GD: find x to minimize $g(x) = x^2$

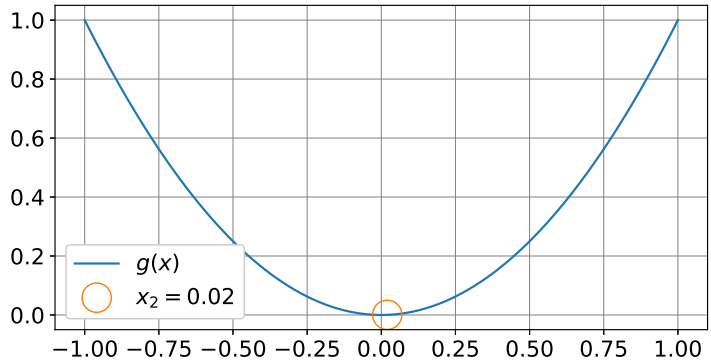
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_1 = 0.1, \quad \nabla_x g(x_1) = 0.2, \quad x_2 = x_1 - 0.4 \nabla_x g(x_1) = 0.02$





Example of GD: find x to minimize $g(x) = x^2$

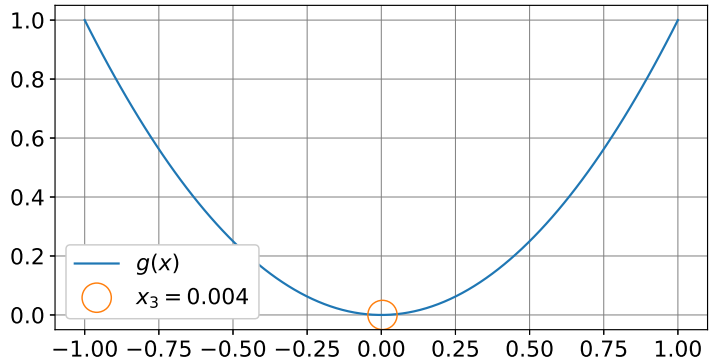
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_2 = 0.02, \quad \nabla_x g(x_2) = 0.04, \quad x_3 = x_2 - 0.4 \nabla_x g(x_2) = 0.004$





Example of GD: find x to minimize $g(x) = x^2$

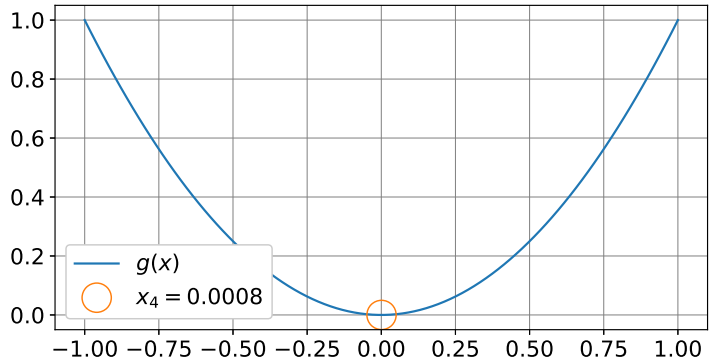
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_3 = 0.004, \quad \nabla_x g(x_3) = 0.008, \quad x_4 = x_3 - 0.3\nabla_x g(x_3) = 0.0008$





Example of GD: find x to minimize $g(x) = x^2$

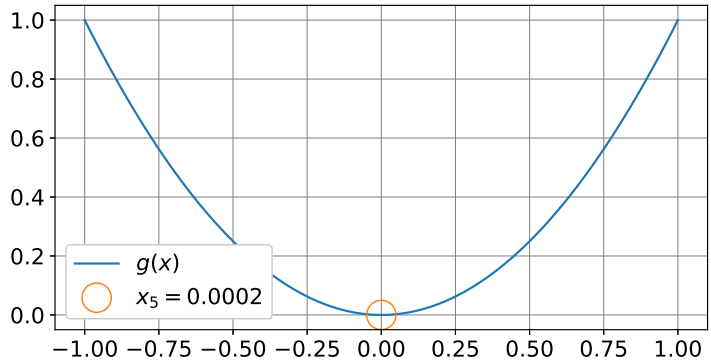
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_4 = 0.0008, \quad \nabla_x g(x_4) = 0.0016, \quad x_5 = x_4 - 0.4 \nabla_x g(x_4) = 0.0002$





Example of GD: find x to minimize $g(x) = x^2$

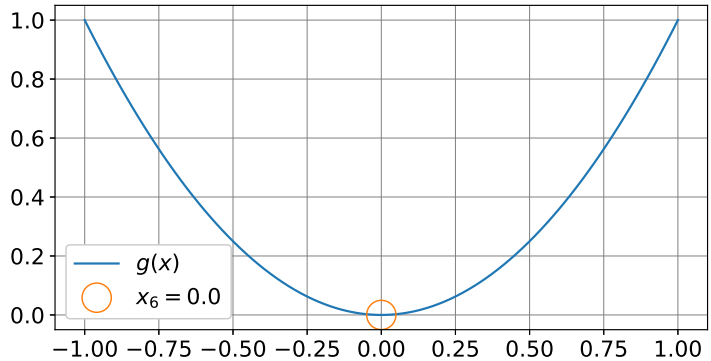
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_5 = 0.0002, \quad \nabla_x g(x_5) = 0.0004, \quad x_6 = x_5 - 0.4 \nabla_x g(x_5) = 0.0000$





Example of GD: find x to minimize $g(x) = x^2$

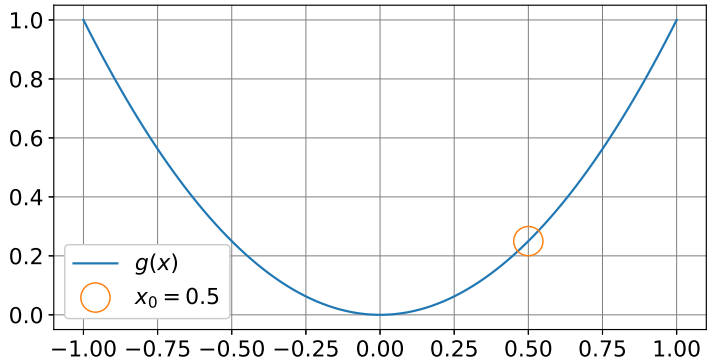
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.4$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$
    - $x_6 = 0.0000, \quad \nabla_x g(x_6) = 0.0000, \quad x_7 = x_6 - 0.4 \nabla_x g(x_6) = 0.0000 = x_6$





What if η is small?

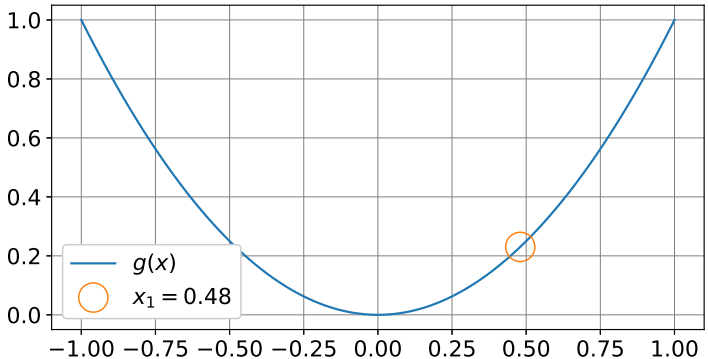
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is small?

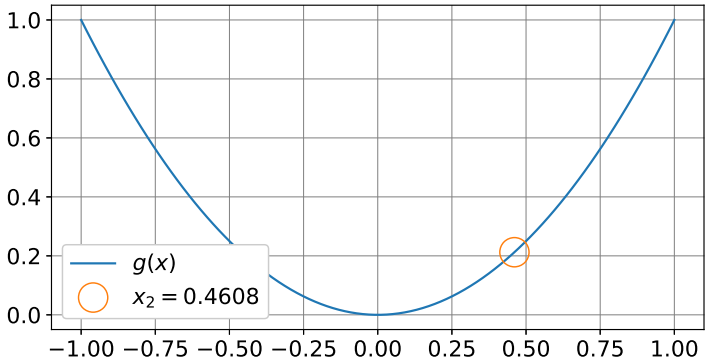
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is small?

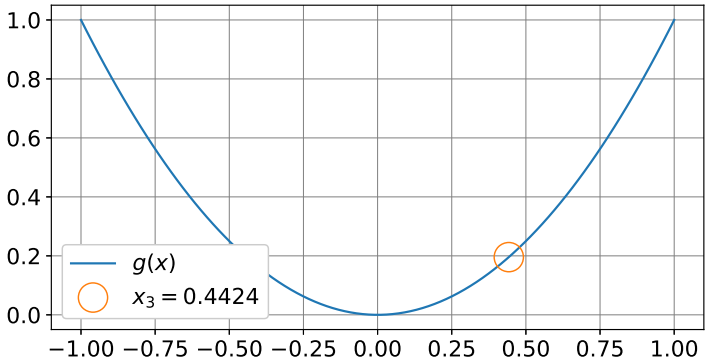
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is small?

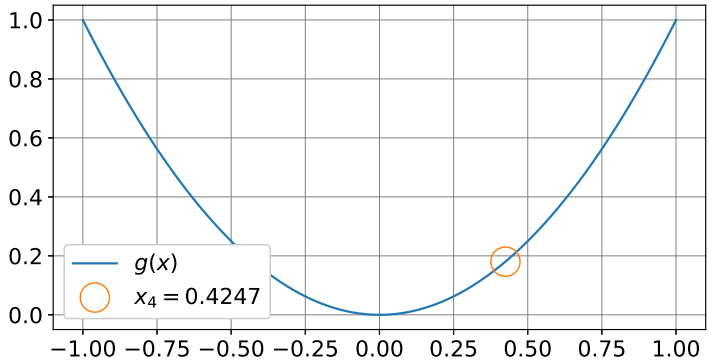
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is small?

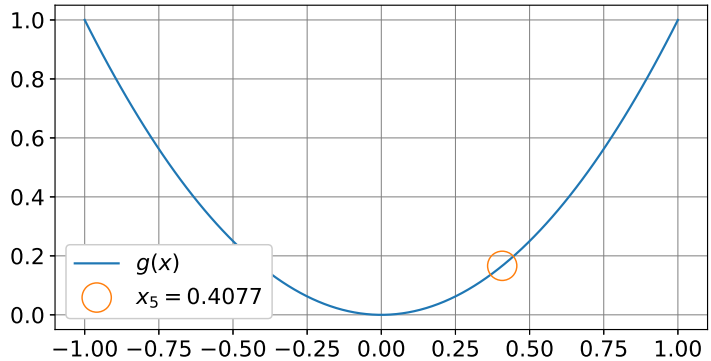
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is small?

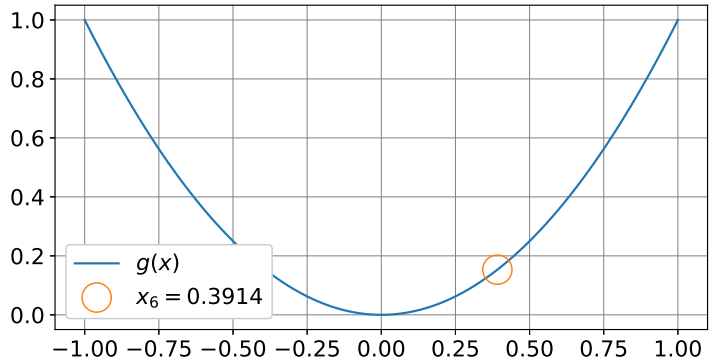
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is small?

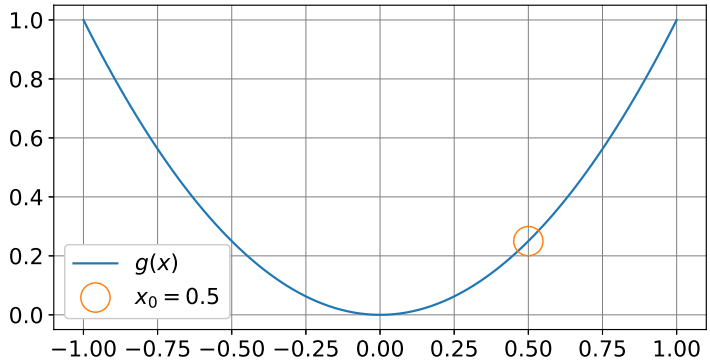
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 0.02$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

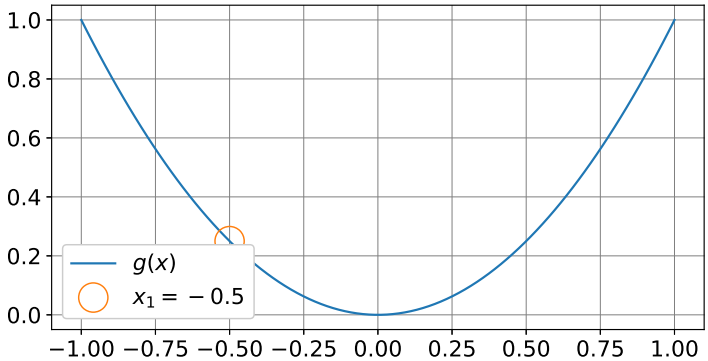
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

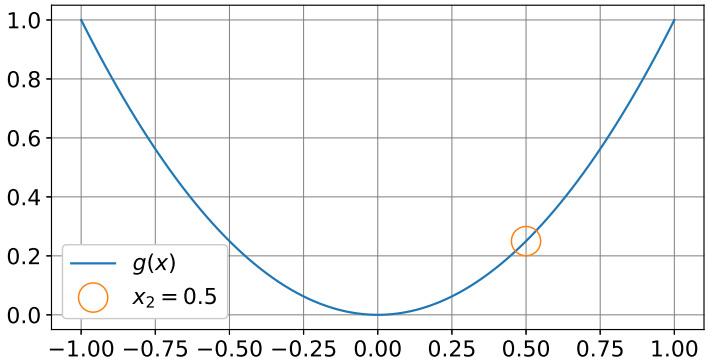
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

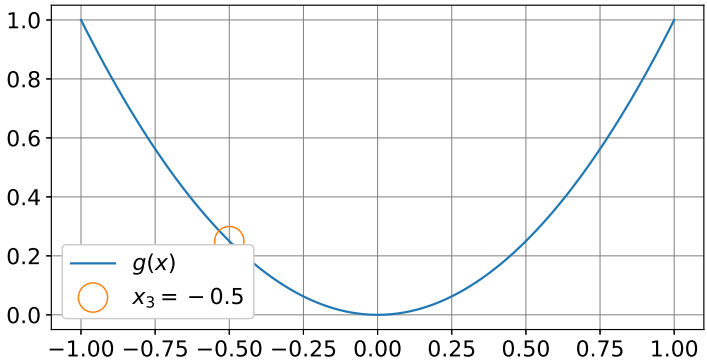
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

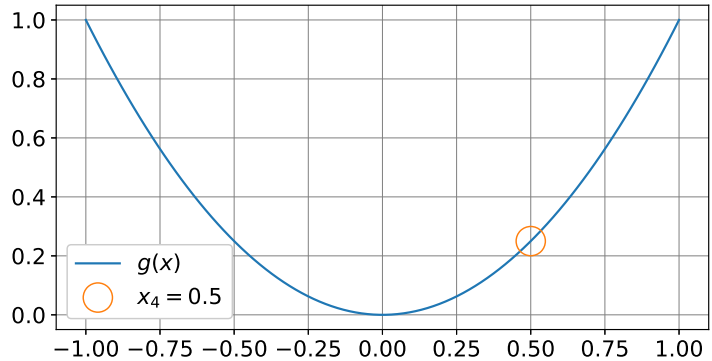
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

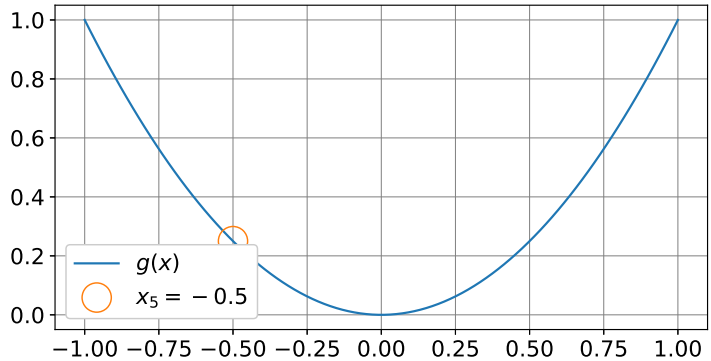
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

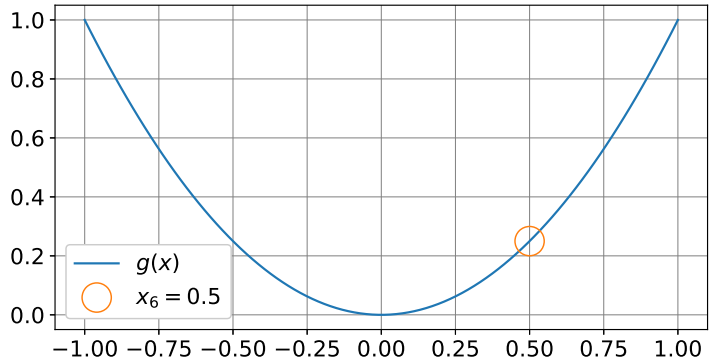
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





What if η is large?

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
  - Gradient $\nabla_x g(x) = 2x$
  - Initialize $x_0 = 0.5$, learning rate $\eta = 1$
  - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k) = x_k - 2\eta x_k$





GD: impact of η

- η is too small:
  - Converge slowly

- η is too large:
  - Overshoot the local
  - Converge slowly to the minimum or may never converge



Exercise of GD

- Find the minimum of function

$$g(\mathbf{x}) = \mathbf{x}^\top \mathbf{x} + [1\ 0] \cdot \mathbf{x} + 5,\ \mathbf{x} \in \mathbb{R}_{1\times 2}$$

  with gradient descent.
- Let $\mathbf{x}_0 = [0.5\ 1]^\top$ and learning rate $\eta = 0.2$:
- What are the values of $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$?
- (Optional) What are the values of $\mathbf{x}_4, \mathbf{x}_5$?



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  - Changing learning rate:



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

1. Decreasing learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta_k \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

$\eta_{k+1} = \eta_k / \alpha$ or $\eta_k - \alpha$ or ...



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

1. Decreasing learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta_k \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

$\eta_{k+1} = \eta_k / \alpha$ or $\eta_k - \alpha$ or ...

Why: early stage - speed up convergence, later stage - refine the solution



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- $M_k = \text{diag}(\sum_{s=0}^{k} \nabla_{\mathbf{w}} C(\mathbf{w}_s) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_s))$ - a diagonal matrix where each diagonal entry is the sum of the element-wise squared gradients (i.e., $\nabla_{\mathbf{w}} C(\mathbf{w}_k) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_k)$)
  $[a_1, a_2, a_3]^\top \odot [b_1, b_2, b_3]^\top = [a_1 b_1, a_2 b_2, a_3 b_3]^\top$
  ε: small constant to prevent division by zero



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- $M_k = \text{diag}(\sum_{s=0}^{k} \nabla_{\mathbf{w}} C(\mathbf{w}_s) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_s))$ - a diagonal matrix where each diagonal entry is the sum of the element-wise squared gradients (i.e., $\nabla_{\mathbf{w}} C(\mathbf{w}_k) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_k)$)
  $[a_1, a_2, a_3]^\top \odot [b_1, b_2, b_3]^\top = [a_1 b_1, a_2 b_2, a_3 b_3]^\top$
  ε: small constant to prevent division by zero
- Element-wise view

$$[\mathbf{w}_{k+1}]_i \leftarrow [\mathbf{w}_k]_i - \frac{\eta}{M_{ii,k} + \varepsilon} [\nabla_{\mathbf{w}} C(\mathbf{w}_k)]_i \ , \quad \text{where } M_{ii,k} = \sum_{s=0}^{k} [\nabla_{\mathbf{w}} C(\mathbf{w}_s)]_i^2$$



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- Pros: adjust the learning rate for based on the historical gradient information

  $\Rightarrow$ gives larger update to $\mathbf{w}_k$ with smaller gradient and smaller update to $\mathbf{w}_k$ with larger gradient.



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- Pros: adjust the learning rate for based on the historical gradient information

  $\Rightarrow$ gives larger update to $\mathbf{w}_k$ with smaller gradient and smaller update to $\mathbf{w}_k$ with larger gradient.
- Cons: learning rate tends to shrink too quickly, leading to slow convergence or getting stuck at suboptimal points (vanishing learning rate).



Variation of gradient descent (GD): different 'gradient'

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Different 'gradient':

1. Momentum-based GD:

$$v_k \leftarrow \beta v_{k-1} + (1-\beta)\nabla_{\mathbf{w}} C(\mathbf{w}_k), \ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta v_k$$

β - momentum factor, $v_0 = \nabla_{\mathbf{w}} C(\mathbf{w}_0)$.



Variation of gradient descent (GD): different 'gradient'

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Different 'gradient':

1. Momentum-based GD:

$$v_k \leftarrow \beta v_{k-1} + (1-\beta)\nabla_{\mathbf{w}} C(\mathbf{w}_k), \ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta v_k$$

β - momentum factor, $v_0 = \nabla_{\mathbf{w}} C(\mathbf{w}_0)$.

Why: maintaining a moving average of past gradients; converging fast but can overshoot the minimum



Variation of gradient descent (GD): different 'gradient'

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Different 'gradient':

2. Nesterov Accelerated Gradient (NAG):

$$v_t \leftarrow \beta v_{t-1} + \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k - \beta v_{t-1}),\ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - v_k$$

'Look ahead'! - anticipate the direction the optimizer will go in the next step



Variation of gradient descent (GD): different 'gradient'

- ⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- ○ Different 'gradient':

2. Nesterov Accelerated Gradient (NAG):

$$v_t \leftarrow \beta v_{t-1} + \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k - \beta v_{t-1}),\ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - v_k$$

'Look ahead'! - anticipate the direction the optimizer will go in the next step

- Pros: faster convergence but slightly more complex
- Cons: slightly more complex than momentum alone



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  - View of dataset:



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- View of dataset:

1. Batch Gradient Descent (BGD):

   Loss function built using the **entire dataset** at each iteration.

Algorithm 3: Batch Gradient Descent (BGD)

Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
while true **do**
  Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  if converge **then**
    return $\mathbf{w}_{k+1}$
  end if
end while



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- View of dataset:

1. Batch Gradient Descent (BGD):

   Loss function built using the **entire dataset** at each iteration.

   - Pros: converges smoothly to the minimum
   - Cons: computationally expensive, memory intensive, not ideal for online learning (hard to update the model incrementally as new data comes)

Algorithm 3: Batch Gradient Descent (BGD)

Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
while true **do**
  Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  if converge **then**
    return $\mathbf{w}_{k+1}$
  end if
end while



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- View of dataset:

2. Stochastic Gradient Descent (SGD):

   Loss function built using **one randomly chosen sample** at each iteration.

Algorithm 4: Stochastic GD (SGD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Randomly choose one sample m
4: Build the loss function $C_m(\mathbf{w})$ with the chosen sample m
5: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_m(\mathbf{w}_k)$
6: **if** converge **then**
7: **return** $\mathbf{w}_{k+1}$
8: **end if**
9: **end while**



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  - View of dataset:
2. Stochastic Gradient Descent (SGD):

   Loss function built using **one randomly chosen sample** at each iteration.

   - Pros: fast updates, suitable for online learning
   - Cons: noisy update, longer convergence time

Algorithm 4: Stochastic GD (SGD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Randomly choose one sample m
4: Build the loss function $C_m(\mathbf{w})$ with the chosen sample m
5: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_m(\mathbf{w}_k)$
6: **if** converge **then**
7: **return** $\mathbf{w}_{k+1}$
8: **end if**
9: **end while**



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- View of dataset:

3. Mini-Batch Gradient Descent (MBGD):

   Loss function built using **one randomly chosen sample** at each iteration.

Algorithm 5: Mini-Batch GD (MBGD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: Divide the entire dataset into small mini-batches
3: **while** true **do**
4: Randomly choose one small mini-batch n
5: Build the loss function $C_n(\mathbf{w})$ with the chosen mini-batch n
6: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_n(\mathbf{w}_k)$
7: **if** converge **then**
8: **return** $\mathbf{w}_{k+1}$
9: **end if**
10: **end while**



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- View of dataset:

3. Mini-Batch Gradient Descent (MBGD):

   Loss function built using **one randomly chosen sample** at each iteration.

- Pros: between SGD and BGD
- Cons: still has noise, hyperparameter tuning (choice of batch size)

Algorithm 5: Mini-Batch GD (MBGD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: Divide the entire dataset into small mini-batches
3: **while** true **do**
4: Randomly choose one small mini-batch n
5: Build the loss function $C_n(\mathbf{w})$ with the chosen mini-batch n
6: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_n(\mathbf{w}_k)$
7: **if** converge **then**
8: **return** $\mathbf{w}_{k+1}$
9: **end if**
10: **end while**



Variation/convergence of gradient descent (GD)

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

  Decreasing learning rate, Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate
- Different 'gradient':

  Momentum-based GD, Nesterov Accelerated Gradient (NAG)
- Design of cost function:

  Batched GD, Stochastic GD, Mini-Batch GD



Variation/convergence of gradient descent (GD)

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

  Decreasing learning rate, Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate
- Different 'gradient':

  Momentum-based GD, Nesterov Accelerated Gradient (NAG)
- Design of cost function:

  Batched GD, Stochastic GD, Mini-Batch GD
- Adam (Adaptive Moment Estimation), Newton
- ...

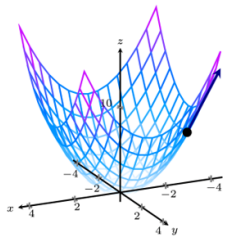
⊙ Book: 'Convex Optimization' by Stephen Boyd and Lieven Vandenberghe

Available at https://stanford.edu/~boyd/cvxbook/



Gradient descent (GD) algorithm: motivation

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \ldots, w_d]^\top$
- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$



  - Gradient $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$ (hard to find the solution of $\nabla_{\mathbf{w}} C(\mathbf{w})$)
  - $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is increasing most rapidly
    $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly

⊙ Hint: Think about the case with $\mathbf{w} \in \mathbb{R}^1$.

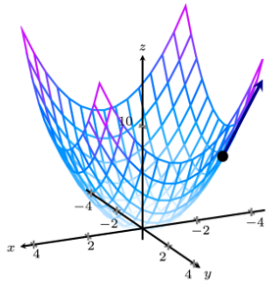
⊙ Book: 'Multivariable Calculus' by James Stewart



Gradient descent (GD) algorithm: basics

Algorithm 6: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
4: **if** converge **then**
5: **return** $\mathbf{w}_{k+1}$
6: **end if**
7: **end while**



◦ According to multi-variable calculus, if η is not too big, then $C(\mathbf{w}_{k+1}) < C(\mathbf{W}_k)$.

⇒ We get better $\mathbf{w}$ after each iteration.



Gradient descent (GD) algorithm: basics

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold
- Limitation: Gradient descent can only find local minimum
  - Because gradient $= \mathbf{0}$ at local minimum

    so $\mathbf{w}$ won't change after that
- Impact of η
  - η is too small:
    - Converge slowly
  - η is too large:
    - Overshoot the local
    - Converge slowly to the minimum or may never converge



Outline

1 Ridge regression

2 Gradient descent

3 Logistic regression

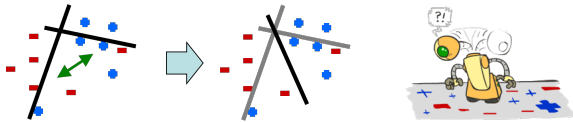


Possible issues for classification



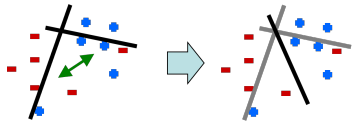
Possible issues for classification

- Noise: data is not separable



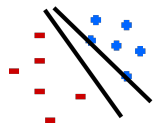
Possible issues for classification

- Noise: data is not separable





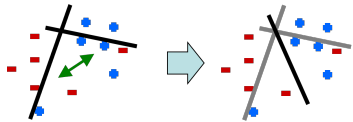
- Mediocre generalization:
  finds a "barely" separating solution





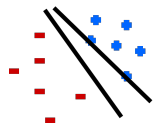
Possible issues for classification

- Noise: data is not separable



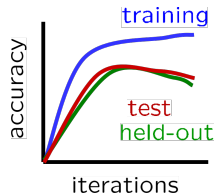


- Mediocre generalization:
  finds a "barely" separating solution



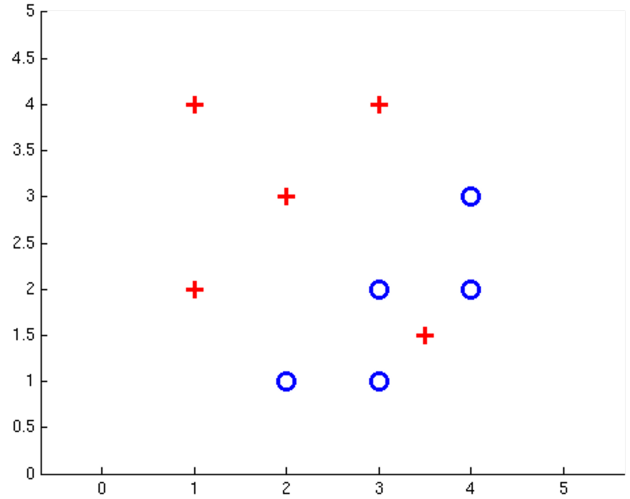


- Overtraining: test / held-out accuracy usually rises, then falls
  - Overtraining is a kind of overfitting





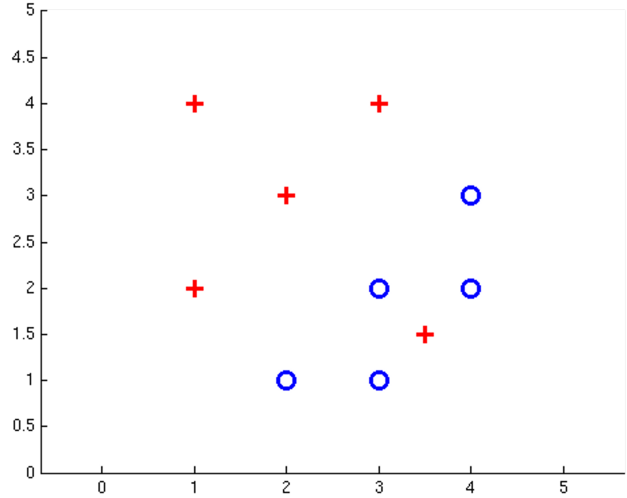
Non-separable case: deterministic decision





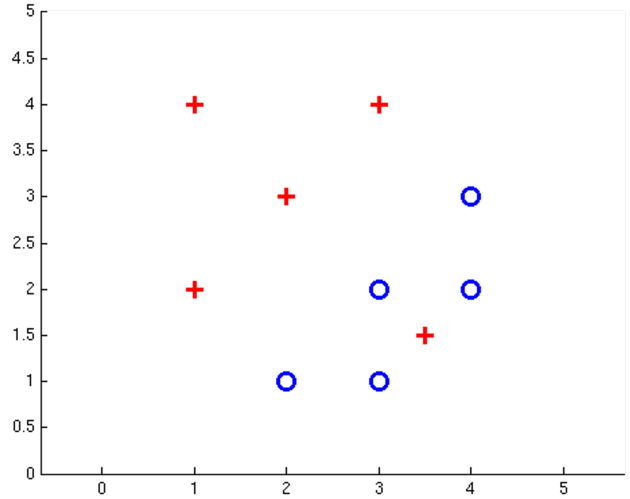
Non-separable case: deterministic decision

Even the best linear boundary makes at least one mistake





Non-separable case: probabilistic decision





Case: predicting the winner

- Deterministic decision: "I predict Team A will beat Team B."
- Probabilistic decision: "I predict Team A has a 55% chance of beating Team B."



Case: predicting the winner

- Deterministic decision: "I predict Team A will beat Team B."
- Probabilistic decision: "I predict Team A has a 55% chance of beating Team B."

- Sometimes we want to model our **confidence** in the labels, which may be noisy.
- Can be done through a **probability distribution** over the labels.



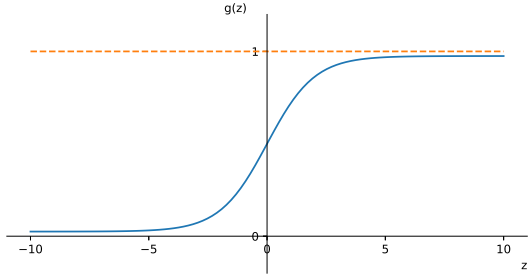
How to get probabilistic decisions?



Logistic regression

Logistic function/sigmoid function:

$$g(z) = \frac{1}{1 + \exp(-z)} \quad z \in \mathbb{R}. \tag{3.1}$$

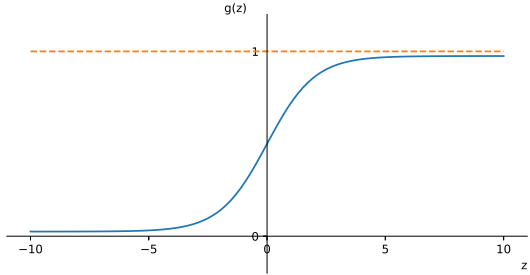




Logistic regression

Logistic function/sigmoid function:

$$g(z) = \frac{1}{1+\exp(-z)} \quad z \in \mathbb{R}. \tag{3.1}$$



Logistic regression: given a sample $\mathbf{x}$ and the parameters $(\boldsymbol{\theta}, \theta_0)$, we set the probability of label $y = 1$ as

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = g(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0) = \frac{1}{1+\exp(-(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0))}. \tag{3.2}$$

This is known as the class conditional probability.



Logistic regression: Why?

One way to derive the form of the logistic function:

- The **log-odds** of the predicted class probabilities should be a linear function of the inputs

$$\log \frac{\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{\Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$$



Logistic regression: Why?

One way to derive the form of the logistic function:

⊙ The **log-odds** of the predicted class probabilities should be a linear function of the inputs

$$\log \frac{\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{\Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$$

- Since $\Pr(y = 1 \mid E) = 1 - \Pr(y = -1 \mid E)$ (probabilities add to 1), we have

$$\log \frac{\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{1 - \Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$$

- Solving this for $\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)$ yields the logistic regression model in (3.2) with logistic function in (3.1).



Logistic regression: Why?

$$\log \frac{\Pr(y=1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{\Pr(y=-1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0 \;\Rightarrow\; \Pr(y=1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1+\exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)}.$$



Samples on the boundary

- Sample $\mathbf{x}$ sits on the decision boundary:

$$\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0 = 0.$$

$\Rightarrow$ We are maximally uncertain about its label:

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = 1/2.$$



Samples on the boundary

- Sample $\mathbf{x}$ sits on the decision boundary:

$$\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0 = 0.$$

$\Rightarrow$ We are maximally uncertain about its label:

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = 1/2.$$

- Logistic regression is known as a generalized linear model because the class conditional probabilities are a function (namely g) of a linear function of the sample $\mathbf{x}$

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)}.$$



Logistic regression: How?



Logistic regression: How?

⊙ Estimate parameters $(\boldsymbol{\theta}, \theta_0)$ with MLE.

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right) = \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}, \tag{3.3}$$

  we want to learn the parameters given the dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)_t\}$.



Logistic regression: How?

⊙ Estimate parameters $(\boldsymbol{\theta}, \theta_0)$ with MLE.

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right) = \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}, \tag{3.3}$$

  we want to learn the parameters given the dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)_t\}$.

- Maximize the probability of seeing the dataset $\mathcal{D}$ over all the parameters $(\boldsymbol{\theta}, \theta_0)$.
- Likelihood of $(\mathbf{x}_t, y_t)$ given $(\boldsymbol{\theta}, \theta_0)$: probability of seeing one sample $(\mathbf{x}_t, y_t)$ given that the current parameters are $(\boldsymbol{\theta}, \theta_0)$ is

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) := \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0)\big). \tag{3.4}$$



Likelihood of dataset

Likelihood of the dataset $\mathcal{D}$: overall likelihood of seeing the dataset given the parameters

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0). \tag{3.5}$$



Likelihood of dataset

Likelihood of the dataset $\mathcal{D}$: overall likelihood of seeing the dataset given the parameters

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0). \tag{3.5}$$

Is this a good formulation?

- Assumption: samples are independent.
- Usually there is little reason that data points are correlated.



Why MLE

- Obtain **maximum likelihood estimates (MLE)** of the parameters by maximizing this conditional likelihood with respect to $\boldsymbol{\theta}$ and θ_0.



Why MLE

- Obtain **maximum likelihood estimates (MLE)** of the parameters by maximizing this conditional likelihood with respect to $\boldsymbol{\theta}$ and θ_0.

- Assumptions:
  - Right model class is selected: e.g., logistic regression model.
  - Certain regularity conditions hold: e.g., $l2$-norm.

    'Likelihood': $L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) + \lambda ||\boldsymbol{\theta}||_2$



Why MLE

- Obtain **maximum likelihood estimates (MLE)** of the parameters by maximizing this conditional likelihood with respect to $\boldsymbol{\theta}$ and θ_0.
- Assumptions:
  - Right model class is selected: e.g., logistic regression model.
  - Certain regularity conditions hold: e.g., $l2$-norm.

    'Likelihood': $L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) + \lambda ||\boldsymbol{\theta}||_2$
- MLE has many **nice** properties (considering $|\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}|$):

$$\hat{\boldsymbol{\theta}}_n \to \boldsymbol{\theta} \text{ as } n \to \infty.$$

  - Consistent/unbiased with $l2$-norm (we will get the right parameter values in the limit of a large number of training examples);
  - Asymptotically normal;
  - Efficient (no other estimator will converge to the correct parameter values faster in the mean squared sense).



Find MLE of parameters: take logarithm

- Take the logarithm of the MLE problem: formulate the following optimization problem:

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \sum_{t=1}^{n} \log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big). \tag{3.6}$$



Find MLE of parameters: take logarithm

- Take the logarithm of the MLE problem: formulate the following optimization problem:

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \sum_{t=1}^{n} \log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big). \tag{3.6}$$

- Convert to minimization:

$$\arg\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) \equiv \arg\min_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} -\log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) \tag{3.7}$$



Find MLE of parameters: take logarithm

- Take the logarithm of the MLE problem: formulate the following optimization problem:

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \sum_{t=1}^{n} \log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big). \tag{3.6}$$

- Convert to minimization:

$$\arg\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) \equiv \arg\min_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} -\log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) \tag{3.7}$$

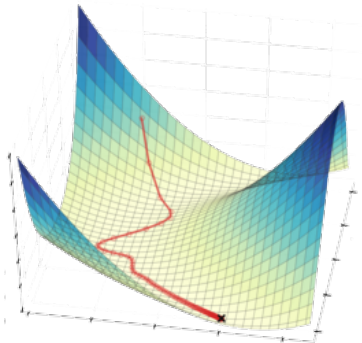
- Equivalent to minimizing:

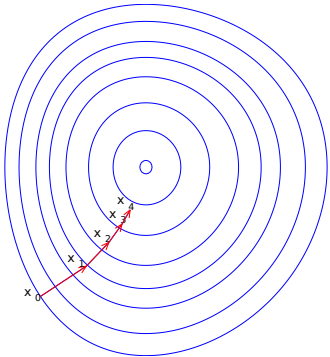
$$\sum_{t=1}^{n} -\log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big) = \sum_{t=1}^{n} \log\big[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\big]. \tag{3.8}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Optimize (3.8) using calculus with a **stochastic gradient descent (SGD)** algorithm.







Find MLE of parameters: stochastic gradient descent (SGD)

- Take the gradients with respect to $\boldsymbol{\theta}$ and θ_0 separately

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}\theta_0}\log\left[1+\exp(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0))\right] &= -y_t\frac{\exp\left(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0)\right)}{1+\exp\left(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0)\right)}\\ &= -y_t\big[1-\Pr(y_t\mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\big], \qquad (3.9)\\ \frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\log\left[1+\exp(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0))\right] &= -y_t\mathbf{x}_t\big[1-\Pr(y_t\mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\big]. \qquad (3.10)\end{aligned}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Take the gradients with respect to $\boldsymbol{\theta}$ and θ_0 separately

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}\theta_0}\log\big[1+\exp(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0))\big] &= -y_t\frac{\exp\big(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0)\big)}{1+\exp\big(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0)\big)}\\ &= -y_t\big[1-\Pr(y_t\mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\big], \qquad (3.9)\\ \frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\log\big[1+\exp(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0))\big] &= -y_t\mathbf{x}_t\big[1-\Pr(y_t\mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\big]. \qquad (3.10)\end{aligned}$$

⊙ **Parameter update:** by selecting training examples at random and moving the parameters in the opposite direction of the derivatives ($\eta > 0$: small learning rate)

$$\begin{aligned}\boldsymbol{\theta} &\longleftarrow \boldsymbol{\theta}+\eta\cdot y_t\mathbf{x}_t\big[1-\Pr(y_t\mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\big] \qquad (3.11)\\ \theta_0 &\longleftarrow \theta_0+\eta\cdot y_t\big[1-\Pr(y_t\mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\big]. \qquad (3.12)\end{aligned}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Setting the gradient to zero is a necessary condition for the optimality of $(\boldsymbol{\theta}^*, \theta_0^*)$

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0, \tag{3.13}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}. \tag{3.14}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Setting the gradient to zero is a necessary condition for the optimality of $(\boldsymbol{\theta}^*, \theta_0^*)$

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0, \tag{3.13}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}. \tag{3.14}$$

- SGD leads to no significant change on average when the gradient of the full objective equals zero.



Find MLE of parameters: stochastic gradient descent (SGD)

- Setting the gradient to zero is a necessary condition for the optimality of $(\boldsymbol{\theta}^*, \theta_0^*)$

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \mathsf{Pr}(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0, \tag{3.13}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \mathsf{Pr}(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}. \tag{3.14}$$

- SGD leads to no significant change on average when the gradient of the full objective equals zero.

⊙ Sum in (3.13): Difference between mistake probabilities associated with positively and negatively labeled examples.
⊙ Optimality of θ_0^*: Mistakes are balanced in this (soft) sense.



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

- Optimality of θ_0^*: vector of optimal prediction errors is orthogonal to the vector of labels.
- Optimality of $\boldsymbol{\theta}^*$: vector of optimal prediction errors is orthogonal to all linear functions of the inputs.



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

⊙ Optimality of θ_0^*: vector of optimal prediction errors is orthogonal to the vector of labels.
⊙ Optimality of $\boldsymbol{\theta}^*$: vector of optimal prediction errors is orthogonal to all linear functions of the inputs.

- Map ± 1 labels to $\{0, 1\}$ with $\tilde{y}_t = (1 + y_t)/2$:

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \big[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0,$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \mathbf{x}_t\big[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}.$$



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

$\odot$ Optimality of θ_0^*: vector of optimal prediction errors is orthogonal to the vector of labels.
$\odot$ Optimality of $\boldsymbol{\theta}^*$: vector of optimal prediction errors is orthogonal to all linear functions of the inputs.

- Map ± 1 labels to $\{0, 1\}$ with $\tilde{y}_t = (1 + y_t)/2$:

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0} \ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t \big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \big[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0,$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}} \ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t \mathbf{x}_t \big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \mathbf{x}_t \big[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}.$$

- Mistake probability: $1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)$
- Prediction error: $\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)$



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

$\odot$ Two optimality conditions: optimal prediction errors are orthogonal to any linear function of the inputs, i.e., for any $(\tilde{\boldsymbol{\theta}}, \tilde{\theta}_0)$,

$$\tilde{\theta}_0 \sum_{t=1}^{n} \left[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\right] + \left\langle \tilde{\boldsymbol{\theta}}, \sum_{t=1}^{n} \mathbf{x}_t \left[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\right] \right\rangle = 0, \quad (3.15)$$

or

$$\sum_{t=1}^{n} \left(\tilde{\theta}_0 + \langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle\right) \left[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\right] = 0. \quad (3.16)$$



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

$\odot$ Vector of optimal prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is no further linearly available information in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to improve the prediction probabilities (or mistake probabilities).



Understand θ^* and θ_0^* (optional)

$\odot$ Vector of optimal prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is no further linearly available information in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to improve the prediction probabilities (or mistake probabilities).

Theorem 3.1

At the optimal solution for the parameter and offset vector $\boldsymbol{\theta}^*, \theta_0^*$, the prediction error vector $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 | \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the data, i.e., for any $\bar{\boldsymbol{\theta}}, \bar{\theta}_0$,

$$\langle \mathbf{z}, \mathbf{e} \rangle = 0, \quad \textit{where} \quad \mathbf{z} := (\langle \bar{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \bar{\theta}_0)_{t=1}^n. \tag{3.17}$$

Furthermore if a prediction vector $\mathbf{e}$ satisfies (3.17), it has the **minimal ℓ_2 norm** *of $(\tilde{y}_t - \Pr(y = 1 | \mathbf{x}_t, \boldsymbol{\theta}, \theta_0))_{t=1}^n$ over all $\boldsymbol{\theta}, \theta_0$.*



Understand θ^* and θ_0^* (optional)

$\odot$ Vector of optimal prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is no further linearly available information in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to improve the prediction probabilities (or mistake probabilities).

Theorem 3.1

At the optimal solution for the parameter and offset vector $\boldsymbol{\theta}^*, \theta_0^*$, the prediction error vector $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 | \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the data, i.e., for any $\bar{\boldsymbol{\theta}}, \bar{\theta}_0$,

$$\langle \mathbf{z}, \mathbf{e} \rangle = 0, \quad \textit{where} \quad \mathbf{z} := (\langle \bar{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \bar{\theta}_0)_{t=1}^n. \tag{3.17}$$

Furthermore if a prediction vector $\mathbf{e}$ satisfies (3.17), it has the **minimal ℓ_2 norm** *of $(\tilde{y}_t - \Pr(y = 1 | \mathbf{x}_t, \boldsymbol{\theta}, \theta_0))_{t=1}^n$ over all $\boldsymbol{\theta}, \theta_0$.*

Hint for proof: Assume $\langle \mathbf{z}, \mathbf{e} \rangle \neq 0$, we can prove $\mathbf{e}' := \mathbf{e} - \dfrac{\langle \mathbf{z}, \mathbf{e} \rangle}{\langle \mathbf{z}, \mathbf{z} \rangle} \mathbf{z}$ satisfies $\|\mathbf{e}'\| < \|\mathbf{e}\|$.



Procedure in logistic regression

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right) = \frac{1}{1+\exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}, \tag{3.3}$$

- Maximize the likelihood of the dataset $\mathcal{D}$

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0). \tag{3.18}$$

- Take the logarithm of the MLE problem

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) \;\Leftrightarrow\; \min_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log\left[1+\exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\right]. \tag{3.19}$$



Procedure in logistic regression

• SGD: Take the gradients with respect to $\boldsymbol{\theta}$ and θ_0 separately and update parameters $\boldsymbol{\theta}$ and θ_0 ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t \big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big] \tag{3.20}$$

$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t \big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big]. \tag{3.21}$$



Regularization in logistic regression

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.

⊙ **Q:** Why?



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.
    Hint: By scaling up the parameters, we make these values larger and larger.

⊙ **Q:** Why?



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.
    Hint: By scaling up the parameters, we make these values larger and larger.

⊙ **Q:** Why?

A: $\log \dfrac{\Pr\left(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0\right)}{\Pr\left(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0\right)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$



Regularization in logistic regression

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t. By scaling up the parameters, we make these values larger and larger.
- Regularization: e.g., ℓ_2-norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^{n} \log\left[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))\right]. \tag{3.22}$$



Multi-class logistic regression

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{3.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{3.23}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.



Multi-class logistic regression

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{3.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{3.23}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

• Without loss of generality, we can assume that one of the classes (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$, which is useful for showing that (3.2) is a special case of (3.27).

◦ **Why?**



[Exercise] Multi-class logistic regression

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{3.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{3.27}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

- **Q:** show that (3.2) is a special case of (3.27).

○ Hint: without loss of generality, we can assume that one of the classes (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$.



Python demo: logistic regression

Canvas: DSAA2011-L4_supervised_c_demo_logistic_reg.pdf



Summary of logistic regression

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t\rangle + \theta_0\right)\right) = \frac{1}{1+\exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t\rangle + \theta_0\right)\right)}, \tag{3.3}$$

- Take the gradients of

$$\log\left[1+\exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\right]$$

with respect to $\boldsymbol{\theta}$ and θ_0 separately and update parameters $\boldsymbol{\theta}$ and θ_0 ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t\left[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\right] \tag{3.24}$$

$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t\left[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\right]. \tag{3.25}$$

- Stochastic gradient descent: update $\boldsymbol{\theta}$ and θ_0 until the termination condition is met.



Summary of logistic regression: regularization

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t. By scaling up the parameters, we make these values larger and larger.
- Regularization: e.g., ℓ_2-norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^{n} \log\left[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))\right]. \tag{3.26}$$



Summary of logistic regression: multi-class classification

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{3.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{3.27}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

• Without loss of generality, we can assume that one of the classes (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$, which is useful for showing that (3.2) is a special case of (3.27).
  ◦ **Why?**



Summary of logistic regression: further exploration

- See the impact of regularization parameter λ
- Try it for multi-class classification

- Find some dataset from Kaggle:
  `https://www.kaggle.com/code/rtatman/datasets-for-regression-analysis`
- Useful Python packages and functions
  `sklearn.linear_model.LogisticRegression`
  `numpy.random`
  `matplotlib.pyplot.scatter`



Thanks for listening

1. Tell us your question/feedback via the QR code.
2. Lab reminder: Th 7-8PM[LA01] / 6-7PM[LA02]

⊙ Slides credit: some slides are adapted from Vincent Y. F. Tan (NUS).

